

Universidad San Sebastián

Sistemas Operativos

Solemne 01 Práctico - Parte 2 - Forma B

Informe final asincrónico

Equipo 07 - Sección NRC 18897

Lenguaje utilizado en la Parte 1: Python

Integrantes

Benjamín Zamora

José Palma

Franco Maripil

Nicolás Portilla

Thomas Márquez

09 de septiembre de 2026

1. Introducción y objetivos

Este informe documenta el traslado de la solución de monitoreo ambiental del Equipo 07 a un ambiente Debian GNU/Linux, como exige la Parte 2 de la Solemne 01 práctica (Forma B): instalación de la máquina virtual, preparación del entorno, ejecución de las dos versiones de la Parte 1 sobre el mismo conjunto de datos, gestor de incidencias y observación de procesos, memoria y sistema de archivos. Todas las cifras se midieron dentro de la máquina virtual en una ÚNICA corrida y quedaron en evidencias/. El documento lo genera scripts/generar_informe.py, que lee esos archivos, los compara con el estado real del árbol entregado y se detiene sin emitir nada si ambas fuentes no coinciden.

1.1 Objetivo de cada componente

- src/secuencial.py: procesar un archivo JSONL completo antes de pasar al siguiente, sin hilos. Define el contrato de validación y es la referencia contra la cual se compara el resultado concurrente.
- src/concurrente.py: procesar el mismo conjunto repartiendo los archivos entre 3 trabajadores threading.Thread mediante una queue.Queue compartida, protegiendo acumuladores globales y bitácora de alertas con mutex threading.Lock.
- src/gestor_incidencias.py: clasificar los informes por cantidad de alertas, resguardar el resumen, separar alertas por indicador, generar el inventario JSON y mantener una bitácora trazable sin detenerse ante entradas defectuosas.

1.2 Flujo del sistema de principio a fin

- Generación de datos: scripts/generar_archivos_entrada.py crea 20 archivos estacion_CODIGO_AAAAMMDD.jsonl en entrada/ con semilla fija 20240908, de modo que la entrada es idéntica en cualquier máquina.
- Ingesta y validación: cada línea JSONL se valida en formato, tipos, rangos y timestamp; la que no cumple se descarta como inválida sin detener el proceso.
- Procesamiento: la versión secuencial recorre los archivos uno tras otro; la concurrente los encola en una queue.Queue y los reparte entre 3 hilos que acumulan resultados locales y sólo actualizan las variables globales dentro de una sección crítica.
- Detección de alertas: cada medición válida se contrasta con los umbrales y la alerta se escribe en alertas/alertas_detectadas.log con el formato archivo;estacion;timestamp;indicador;valor;umbral.
- Salida: un informe por archivo en salida/informe_*.txt y el consolidado salida/resumen_ambiental.txt.
- Gestión de incidencias: los informes se MUEVEN a gestion_ambiental/sin_alertas/, con _alertas/ o criticas/ según sus alertas; el resumen se COPIA como resumen_resguardado.txt; las alertas se separan por indicador; se generan inventario_ambiental.json y logs/gestion_ambiental.log. Por eso salida/ queda sólo con el resumen: ver 5.4.

1.3 Reglas de validación y umbrales de alerta

Campo	Rango válido	Indicador	Condición de alerta
Temperatura	-20.0 C a 60.0 C	Temperatura	mayor o igual a 35.0 C
Humedad	0.0 % a 100.0 %	Humedad	menor o igual a 20.0 %
PM2.5	mayor o igual a 0.0 ug/m3	PM2.5	mayor o igual a 35.0 ug/m3
Ruido	0.0 dB a 140.0 dB	Ruido	mayor o igual a 75.0 dB
Timestamp	formato AAAA-MM-DDTHH:MM:SS		

Ambas versiones comparten estas constantes, y ésa es la razón de fondo por la cual sus métricas deben coincidir hasta el último decimal: la concurrencia cambia el orden en que se hace el trabajo, no el criterio con que se decide.

2. Instalación de Debian 13 en la máquina virtual

2.1 Descarga y verificación del ISO

Se descargó la imagen de instalación por red debian-13.6.0-amd64-netinst.iso (755 MB) desde el sitio oficial de Debian y se verificó su integridad comparando su SHA256 con el publicado en SHA256SUMS, lo que garantiza que el medio no fue alterado ni quedó truncado. La salida es la del anfitrión y está archivada en evidencias/hyperv/configuracion-vm-hyperv.txt:

```
PS> Get-FileHash debian-13.6.0-amd64-netinst.iso -Algorithm SHA256
Hash calculado : 65273beed27b2df543b68b65630ba525cfbad8df2b12035732b2dff87d6664e7
Hash publicado : 65273beed27b2df543b68b65630ba525cfbad8df2b12035732b2dff87d6664e7
Fuente: SHA256SUMS
Coincide      : SI
```

2.2 Declaración del cambio de hipervisor respecto del hito

Declaración explícita. En el hito el equipo planificó usar Oracle VirtualBox y la implementación final se hizo sobre Microsoft Hyper-V (máquina de Generación 1, arranque BIOS). Se hace constar porque afecta lo declarado en el hito; la pauta admite la situación, ya que la instalación "puede hacerlo en VirtualBox, VMware u otro hipervisor". Los recursos comprometidos se respetaron exactamente, sin rebajar ninguno.

Recurso	Mínimo de la pauta	Comprometido en el hito	Implementado y verificado
Hipervisor	VirtualBox, VMware u otro	Oracle VirtualBox	Microsoft Hyper-V, Generación 1
RAM	2 GB	4 GB	4 GB fijos (memoria dinámica: False); 3.8Gi útiles según free -h
Procesadores virtuales	2	4	4 vCPU sobre Intel(R) Core(TM) i9-14900
Disco virtual	20 GB	25 GB	25 GB Dynamic (ocupa 3 GB reales en el anfitrión)
Red	NAT	NAT con conectividad	Default Switch (Internal), IP 172.22.205.178/20 por DHCP
Sistema	Debian 13, 64 bits	Debian 13, 64 bits	Debian GNU/Linux 13 (trixie), versión 13.6

El propio sistema instalado confirma sobre qué hipervisor corre, de modo que la declaración es verificable y no depende de la palabra del equipo:

```
$ systemd-detect-virt ; lscpu | grep 'Hypervisor vendor' ; uname -a
microsoft
Hypervisor vendor:                Microsoft
Linux debian-so-equipo07 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29) x86_64 GNU/Linux
```

De la etapa previa sobre Oracle VirtualBox no se conserva ninguna captura en el entregable, y por eso esta subsección no lleva figura: las imágenes de esa etapa correspondían a otra máquina, con otro usuario y otro nombre de host, de modo que presentarlas habría contradicho lo que documenta el resto del informe. La única evidencia gráfica de instalación es la de la máquina definitiva sobre Microsoft Hyper-V (máquina de Generación 1, arranque BIOS) (2.3 y 2.4); los recursos comprometidos en el hito ya quedaron verificados arriba contra los cmdlets del hipervisor.

2.3 Instalación, usuario y particionado del disco virtual

Las capturas de esta sección y de la siguiente son de la instalación REAL de la máquina definitiva sobre Hyper-V. Se creó de Generación 1, con firmware BIOS heredado, y su orden de arranque (IDE , CD , LegacyNetworkAdapter , Floppy) explica que el instalador arranque en modo BIOS y no UEFI. Se creó el usuario sin privilegios equipo07, con acceso a sudo, y el host quedó como debian-so-equipo07, de modo que cualquier salida de este informe se atribuye sin ambigüedad a la máquina del equipo.

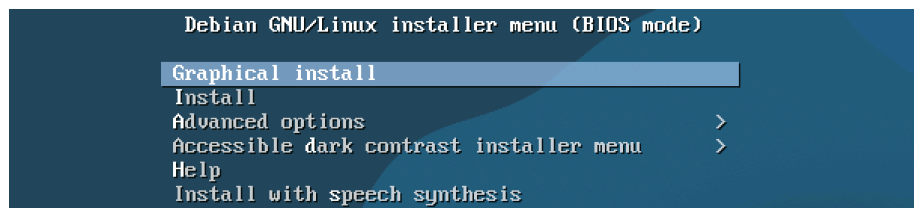


Figura 1. Menú del instalador de Debian 13 en modo BIOS, en la máquina de Generación 1 de Hyper-V. Instalación real de la máquina definitiva.

```
boot: install auto=true priority=critical interface=auto hostname=deb_
```

Figura 2. Línea de arranque del instalador con el archivo de preconfiguración (preseed), que fija idioma, teclado, zona horaria, particionado guiado y el conjunto mínimo de tareas. Usar preseed hace la instalación reproducible: el mismo archivo produce la misma máquina.

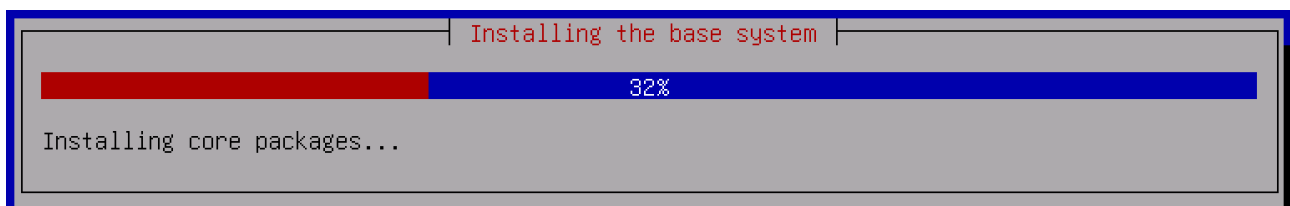


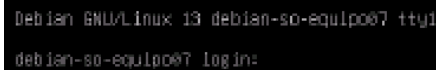
Figura 3. Instalación del sistema base: descarga e instalación de paquetes desde la réplica de red. La etapa final del instalador -gestor de arranque y copia de la configuración de red al sistema instalado- es la misma pantalla de progreso y está archivada en evidencias/fotos/instalacion-hyperv/04-instalacion-final.png.

El particionado se aplicó sobre el disco de 25 GB con el esquema guiado "todo en una partición":

```
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
fd0          2:0    1    4K  0 disk
sda          8:0    0   25G  0 disk
|-sda1       8:1    0  23.7G  0 part /
|-sda2       8:2    0    1K  0 part
|-sda5       8:5    0   1.3G  0 part [SWAP]
sr0         11:0    1 1024M  1 rom
```

- sda1 (23.7 GB, ext4) es la partición raíz montada en /; sda2 es sólo el contenedor lógico de la extendida y sda5 (1.3 GB) es el área de intercambio, que el núcleo usa para descargar páginas cuando la memoria física escasea.
- sr0 es la unidad óptica virtual desde la que se montó el ISO de instalación; terminada la instalación el ISO fue expulsado, como consta en la salida de Get-VMdvdDrive del anfitrión.

2.4 Primer inicio y conectividad



```
Debian GNU/Linux 13 debian-so-equip007 tty1
debian-so-equip007 login:
```

Figura 4. Primer inicio del sistema recién instalado: el núcleo llega a la consola tty1 y presenta el indicador de acceso del equipo debian-so-equip007. La captura muestra el indicador de acceso, no la sesión ya iniciada; que el acceso con el usuario equip007 funciona lo demuestra el resto del informe, cuyas salidas fueron obtenidas en esa sesión.

eth0 obtiene su dirección por DHCP desde el conmutador Default Switch y la ruta por omisión apunta a 172.22.192.1. El Default Switch de Hyper-V provee NAT con DHCP hacia la red 172.22.192.0/20. El anfitrión ve la misma dirección (172.22.205.178, MAC 00155D647F07), lo que cierra la trazabilidad entre lo que declara Hyper-V y lo que observa el huésped. Con esa configuración la máquina alcanza las réplicas de Debian, como confirman la resolución de deb.debian.org y las líneas "Hit" de apt de la sección siguiente.

```
$ ip -4 -br addr show ; ip route ; getent hosts deb.debian.org
lo                UNKNOWN      127.0.0.1/8
eth0              UP           172.22.205.178/20
default via 172.22.192.1 dev eth0 proto dhcp src 172.22.205.178 metric 1002
172.22.192.0/20 dev eth0 proto dhcp scope link src 172.22.205.178 metric 1002
2a04:4e42:34::644 debian.map.fastlydns.net deb.debian.org
```

3. Preparación del ambiente

El punto 2.a de la pauta exige actualizar el sistema antes de trabajar. La salida confirma que los tres orígenes de paquetes están accesibles (trixie, trixie-security y trixie-updates) y que el sistema quedó al día.

```
$ sudo apt update ; sudo apt upgrade -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.

[...]
Calculating upgrade...
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

El punto 2.b pide instalar el entorno del lenguaje usado en la Parte 1. El proyecto está en Python y Debian 13 ya trae Python 3.13.5 en el sistema base, de modo que no hubo que compilar ni agregar repositorios externos. El proyecto usa exclusivamente la biblioteca estándar de Python: sin pip, sin entorno virtual y sin archivo de requisitos, decisión deliberada porque hace reproducible la ejecución incluso sin red. Los cuatro módulos compilan sin errores.

```
$ python3 --version ; apt list --installed | grep -E 'python3|openssh-server|time'
$ python3 -m py_compile src/*.py scripts/*.py # sin salida: los cuatro modulos compilan

Python 3.13.5
openssh-server/stable,now 1:10.0p1-7+deb13u4 amd64 [installed]
python3-venv/stable,now 3.13.5-1 amd64 [installed]
python3/stable,now 3.13.5-1 amd64 [installed]
time/stable,now 1.9-0.2 amd64 [installed]
```

4. Ejecución de la Parte 1 en Debian

El punto 2.c exige ejecutar ambas versiones sobre el mismo conjunto de archivos .jsonl y comprobar que seis métricas coinciden. Para que "el mismo conjunto" sea verificable, la entrada se genera con semilla fija y se le calcula una suma de verificación antes de cada corrida.

```
$ cd /home/equipo07/solemne_so_equipo07
$ python3 scripts/generar_archivos_entrada.py
$ cat entrada/*.jsonl | sha256sum
$ time python3 src/secuencial.py
$ time python3 src/concurrente.py

Archivos .jsonl generados: 20
d4f2b512dd71fd789fe9b2a3c1d0d69ee1bf3f4e8d7c99656dd113dad89b68f2 -

secuencial  real 0m0.019s  user 0m0.016s  sys 0m0.004s  (tiempo interno 0.005 s, 1 trabajador)
concurrente real 0m0.023s  user 0m0.024s  sys 0m0.000s  (tiempo interno 0.007 s, 3 trabajadores)
```

4.1 Las seis métricas exigidas por la pauta

Métrica	Secuencial	Concurrente	Coincide
Archivos procesados	20	20	SI
Mediciones validas	300	300	SI
Mediciones invalidas	20	20	SI
Alertas totales	79	79	SI
PM2.5 maximo global	177.87 ug/m3	177.87 ug/m3	SI
Ruido maximo global	109.77 dB	109.77 dB	SI

Métricas comparadas: 6 de 6. TODAS LAS METRICAS EXIGIDAS COINCIDEN. El resumen consolidado agrega 320 líneas leídas, temperatura promedio 23.46 °C, humedad promedio 54.85 % y estación con más alertas STG04. El entregable conserva el resumen de la corrida concurrente en salida/resumen_ambiental.txt.

4.2 Comprobaciones adicionales de consistencia

Seis totales podrían coincidir por casualidad si dos errores se compensaran, de modo que se agregaron dos comprobaciones más exigentes que las que pide la pauta: los 20 informes individuales comparados uno por uno, con 0 diferencias, y el contenido completo de la bitácora de alertas, con 79 alertas en el secuencial y 79 en el concurrente. El orden de escritura del concurrente varía por diseño, porque 3 hilos vuelcan sus búferes en momentos distintos; por eso el volcado final se ordena por nombre de archivo. IDENTICO BYTE A BYTE entre ambas versiones.

Resultado incómodo que se declara tal cual se midió: la concurrente fue más LENTA que la secuencial en este conjunto (0m0.019s frente a 0m0.023s de tiempo real; 0.005 s frente a 0.007 s de tiempo interno). La explicación está en 8.1.

```
== Parte 1 en Debian: secuencial vs concurrente ==

METRICA                SECUENCIAL          CONCURRENTE          OK
-----
Archivos procesados    20                  20                  SI
Mediciones validas     300                 300                 SI
Mediciones invalidas   20                  20                  SI
Alertas totales        79                  79                  SI
PM2.5 maximo global    177.87 ug/m3        177.87 ug/m3        SI
Ruido maximo global    109.77 dB           109.77 dB           SI
-----

Informes individuales identicos entre versiones: 20 de 20
alertas_detectadas.log: 79 lineas, identico byte a byte

debian-so-equipo07 | Debian 13.6 | Python 3.13.5
```

Figura 5. Comprobación de consistencia en la consola de la máquina virtual: las seis métricas coinciden, los 20 informes individuales son idénticos y la bitácora de alertas tiene 79 alertas en ambas versiones.

5. Gestor de incidencias

El punto 2.d enumera trece requisitos para src/gestor_incidencias.py. La tabla los recorre uno por uno con la evidencia que los respalda; todas las cifras provienen del estado real del proyecto y de los archivos de evidencia, y el generador comprueba que ambas fuentes coincidan antes de escribir esta página.

N	Requisito de la pauta	Evidencia obtenida
1	Detectar informes informe_CODIGO_AAAAMMDD.txt	Patrón ^informe_[A-Za-z0-9]+_d{8}(_vld+)?\.txt\$; 20 informes detectados
2	Leer la cantidad de alertas de cada informe	Línea "Alertas detectadas: N" de cada informe; suma 79 alertas
3	Mover informes con 0 alertas a sin_alertas/	Rama implementada; queda en 0 informes (ver 5.3) y se demostró con informe_CTL00_20240101.txt

N	Requisito de la pauta	Evidencia obtenida
4	Mover informes con 1 a 3 alertas a con_alertas/	8 informes en gestion_ambiental/con_alertas/
5	Mover informes con 4 o más alertas a criticas/	12 informes en gestion_ambiental/criticas/
6	Evitar sobrescritura mediante nombres _v1, _v2	Tres corridas acumulan 61 informes, 40 de ellos con sufijo _v1 o _v2, sin pérdidas (ver 7.3)
7	Copiar el resumen como resumen_resguardado.txt	Copia de 448 bytes, idéntica al original que permanece en salida/
8	Leer alertas/alertas_detectadas.log	79 líneas del formato archivo;estacion;timestamp;indicador;valor;umbral
9	Separar alertas por indicador	temperatura 21, humedad 17, pm25 20, ruido 21
10	Registrar alertas inválidas o desconocidas	4 anomalías registradas al inyectar fallas (ver 7.2)
11	Generar inventario_ambiental.json	Archivo de 3155 bytes, JSON valido con 8 claves de primer nivel
12	Generar logs/gestion_ambiental.log	Bitácora de 28 líneas y 2442 bytes, con marca de tiempo por evento
13	Controlar al menos un error sin detenerse	Con cuatro fallas el gestor terminó con código 0 y conservó las 79 alertas (ver 7.2)

5.1 Estructura generada

```
$ find gestion_ambiental logs -type d | sort
gestion_ambiental
gestion_ambiental/alertas_por_indicador
gestion_ambiental/alertas_por_indicador/humedad
gestion_ambiental/alertas_por_indicador/pm25
gestion_ambiental/alertas_por_indicador/ruido
gestion_ambiental/alertas_por_indicador/temperatura
gestion_ambiental/con_alertas
gestion_ambiental/criticas
gestion_ambiental/sin_alertas
logs
```

Bajo gestion_ambiental/ quedaron 27 archivos: los 26 que produjo el gestor -los 20 informes clasificados, los cuatro registros por indicador, el inventario y el resumen resguardado, los mismos que registra la evidencia- más el marcador sin_alertas/.gitkeep, con el que el control de versiones conserva una carpeta que quedó legítimamente vacía (5.3). El gestor terminó con código 0 en su única ejecución sobre el árbol entregado.

```
gestion_ambiental/alertas_por_indicador/pm25
gestion_ambiental/alertas_por_indicador/ruido
gestion_ambiental/alertas_por_indicador/temperatura
gestion_ambiental/con_alertas
gestion_ambiental/criticas
gestion_ambiental/sin_alertas

sin_alertas/      0 informes
con_alertas/      8 informes
criticas/         12 informes

Alertas por indicador:
temperatura  21
humedad      17
pm25         20
ruido        21
SUMA         79 (= alertas_detectadas.log)

ls -lah gestion_ambiental/*.json gestion_ambiental/*.txt
-rw-rw-r-- 1 equipo07 equipo07 3.1K Sep  9 23:29 gestion_ambiental/inventario_ambiental.json
-rw-rw-r-- 1 equipo07 equipo07 448 Sep  9 23:29 gestion_ambiental/resumen_resguardado.txt
```

Figura 6. Estructura generada por el gestor y cuadratura de las alertas por indicador, en la máquina virtual.

5.2 Inventario y cuadratura de las alertas

El valor del inventario no está en existir sino en cuadrar: la suma de las alertas por indicador debe ser igual al total del inventario, al número de líneas de la bitácora de alertas y al total del resumen consolidado. El generador verifica esa igualdad y además que el número leído en el repositorio sea el mismo de la evidencia congelada; si alguna comprobación falla, se detiene.

Fuente	Alertas	Fuente	Alertas
alertas_por_indicador/temperatura	21	Suma de los cuatro indicadores	79
alertas_por_indicador/humedad	17	alertas/alertas_detectadas.log	79
alertas_por_indicador/pm25	20	inventario_ambiental.json	79
alertas_por_indicador/ruido	21	salida/resumen_ambiental.txt	79

```
$ python3 -m json.tool gestion_ambiental/inventario_ambiental.json
```

```
{
  "resumen": {
    "total_informes": 20,
    "informes_por_categoria": {
      "sin_alertas": 0,
      "con_alertas": 8,
      "criticas": 12
    },
    "total_alertas": 79,
    "anomalias_registradas": 0,
    "errores_de_proceso": 0
  },
  "informes": { [...] },
  "alertas_por_indicador": { [...] },
  "total_alertas": 79,
  "anomalias": { [...] },
  "resumen_resguardado": {
    "existe": true,
    "ruta": "gestion_ambiental/resumen_resguardado.txt"
  },
  "bitacora": "logs/gestion_ambiental.log",
  "marca_de_ejecucion": "2026-09-09 23:29:11"
}
```

total_alertas aparece dos veces porque el inventario lo declara dentro de resumen y también en la raíz; ambos valores son el mismo número que cuadra la tabla anterior, y la bitácora cierra la corrida declarando ese mismo par:

```
$ grep 'Inventario generado' logs/gestion_ambiental.log
```

```
[2026-09-09 23:29:11] Inventario generado en gestion_ambiental/inventario_ambiental.json: 20 informes, 79 alertas.
```

5.3 Por qué sin_alertas/ quedó vacía

La carpeta sin_alertas/ contiene 0 informes y no es un defecto del gestor: la pauta de la Parte 1 exige que cada archivo de entrada produzca al menos una alerta, de modo que ningún informe puede tener cero y la rama queda legítimamente vacía. La rama existe y funciona: con el informe de control informe_CTL00_20240101.txt, construido con cero alertas, el gestor lo movió a sin_alertas/ y la clasificación quedó en sin_alertas/ 1, con_alertas/ 8, criticas/ 12. Esa demostración se hizo sobre una COPIA, por lo que el árbol entregado conserva sus 0 informes en esa rama.

5.4 Por qué salida/ sólo conserva el resumen

En el árbol entregado, salida/ contiene sólo resumen_ambiental.txt y LEEME.txt. No falta nada: los 20 informes individuales SÍ se generaron ahí (src/concurrente.py) y el gestor los MOVIÓ a sus carpetas de clasificación, que es lo que exigen los puntos 3, 4 y 5 de la pauta. Mover y no copiar se explica en 7.1. Antes de ejecutar el gestor se respaldó salida/ completo, de modo que el estado que dejó la Parte 1 también es auditable:

- salida/resumen_ambiental.txt (448 bytes): el consolidado que la pauta pide mantener en su lugar.
- salida/LEEME.txt: explica esta situación dentro del propio entregable e indica cómo regenerar los informes.
- evidencias/salida_parte1/: 21 archivos, es decir los 20 informes individuales más el resumen, tal como los dejó la Parte 1 antes de la clasificación.

6. Observación de procesos y del sistema de archivos

Nota metodológica sobre la carga amplificada. El conjunto oficial de 20 archivos se procesa en milisegundos y ps no alcanza a tomar una muestra útil. Por eso la observación se hizo en dos partes: (A) el consumo de la corrida OFICIAL con /usr/bin/time -v, que no necesita muestreo; y (B) la observación en vivo con ps sobre una CARGA AMPLIFICADA de 4000 archivos (64000 líneas), los mismos archivos oficiales replicados con otras fechas válidas. Ambas partes se ejecutan sobre COPIAS del proyecto, para no alterar el árbol entregable. Toda métrica entregable sale del conjunto oficial; la carga amplificada sólo hace que el proceso viva lo suficiente para observarlo (270 muestras).

6.1 Consumo de recursos de la corrida oficial

```
$ /usr/bin/time -v python3 src/concurrente.py
```

```
Command being timed: "python3 src/concurrente.py"
User time (seconds): 0.01
System time (seconds): 0.00
Percent of CPU this job got: 100%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.02
Maximum resident set size (kbytes): 12964
Voluntary context switches: 149
Involuntary context switches: 1
File system inputs: 0
File system outputs: 184
Exit status: 0
```

- Percent of CPU 100% sobre centésimas de segundo: con sólo 20 archivos el proceso apenas alcanza a repartir trabajo antes de terminar.
- Maximum resident set size 12964 KB (unos 12.7 MB de memoria física): el costo del intérprete de Python más los acumuladores del programa.
- 149 cambios de contexto voluntarios y 1 involuntarios. Los voluntarios son el proceso cediendo la CPU por su cuenta al esperar entrada/salida o un mutex; los involuntarios son el planificador expulsándolo al agotar su cuanto. 184 bloques escritos corresponden a los informes, el resumen y la bitácora de alertas.

6.2 Identificación del proceso concurrente

```
$ ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,etime,cmd -p <PID>
```

```
PID    PPID  STAT  %CPU  %MEM   RSS    VSZ  NLWP   ELAPSED CMD
10043  10018 Rl    75.0   0.4  18824 249776 4      00:00 python3 src/concurrente.py
```

Campo	Valor	Qué significa en este proceso concreto
PID	10043	Identificador que el núcleo asignó al proceso que ejecuta python3 src/concurrente.py; es único mientras el proceso vive.
PPID	10018	Proceso padre: 10018 Ss bash /tmp/rsh_13-procesos-recaptura.sh. Es el intérprete de órdenes desde el que se lanzó la ejecución.
STAT	Rl	Rl: en el instante de la muestra el proceso estaba ejecutándose en CPU; la l final indica que es multihilo. En otras muestras aparece S, durmiendo en espera interrumpible (entrada/salida o un mutex).
%CPU	75.0 (máx. 103)	ps lo promedia sobre toda la vida del proceso, y esta muestra aún arrastra el arranque. El trabajo simultáneo en más de un núcleo lo prueba el máximo observado, 103: un proceso de un solo hilo no pasa de 100 %.
%MEM	0.4	Fracción de los 4 GB de memoria física ocupada por el proceso; es baja porque el programa procesa por flujo y no carga todo en memoria.
RSS	18824 KB	Memoria residente: la parte del proceso efectivamente cargada en RAM, que es lo que cuesta de verdad en memoria física.
VSZ	249776 KB	Memoria virtual reservada, unas trece veces el RSS: incluye bibliotecas compartidas, pilas de hilos y regiones que nunca se tocaron, por lo que no debe leerse como consumo real.
NLWP	4	Hilos vivos: 1 principal + 3 trabajadores, exactamente los que declara el programa.

6.3 Hilos y vista del núcleo

```
$ ps -L -o pid,tid,stat,%cpu,comm -p 10043
$ grep -E 'State|Threads|VmRSS' /proc/10043/status
```

```
PID      TID  STAT  %CPU  COMMAND
10043    10043 Sl    75.0  python3
10043    10108 Sl    0.0  python3
10043    10109 Sl    0.0  python3
10043    10111 Rl    0.0  python3
Name:     python3
State:    S (sleeping)
Tgid:     10043
Pid:      10043
PPid:     10018
VmSize:   249776 kB
VmRSS:    18908 kB
Threads:  4
```

Los cuatro identificadores de hilo comparten el mismo PID pero tienen TID distintos: el primero coincide con el PID y es el hilo principal, que encola y espera; los otros 3 consumen de la cola. El núcleo confirma Threads: 4 y VmRSS: 18908 kB, coherente con ps, y State: S (sleeping). Que el estado sea "sleeping" en una muestra y "running" en otra no es contradictorio: los hilos alternan entre cálculo y espera por entrada/salida.

6.4 Evolución del proceso durante la ejecución

```
$ ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,etime,cmd -p <PID> # repetido sobre el mismo PID
```

```
PID    PPID  STAT  %CPU  %MEM   RSS    VSZ  NLWP   ELAPSED CMD
10043  10018 R    0.0   0.1   7036  19344  1      00:00 python3 src/concurrente.py
10043  10018 Sl   100   0.4  19368 249776  4      00:00 python3 src/concurrente.py
10043  10018 Sl   102   0.4  19964 250800  4      00:00 python3 src/concurrente.py
10043  10018 Sl   103   0.5  20876 251824  4      00:00 python3 src/concurrente.py
10043  10018 R    103   0.5  21600 251776  1      00:00 python3 src/concurrente.py
```

La primera muestra fue tomada antes de que arrancaran los trabajadores: un solo hilo (NLWP 1), 0.0 % de CPU y 6.9 MB residentes. En las siguientes NLWP sube a 4, el %CPU pasa de 100 y el RSS crece de forma sostenida a medida que se llenan los acumuladores; en la última los trabajadores ya terminaron y NLWP vuelve a 1 con el %CPU acumulado en 103. Esa progresión es la concurrencia hecha visible.


```

ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,cmd -p PID
PID  PPID STAT  %CPU %MEM  RSS   VSZ  NLWP CMD
1197  1185 SI    107  0.5 20472 250808 4 python3 src/con
currente.py

%CPU maximo observado con los 4 hilos vivos: 107
(por encima de 100 = trabajo simultaneo en varios nucleos)

Hilos vivos (ps -L): 1 principal + 3 trabajadores
  PID    TID STAT COMMAND
  1197    1197 SI    python3
  1197    1256 SI    python3
  1197    1257 RI    python3
  1197    1258 SI    python3
State:  S (sleeping)
Pid:    1197
PPid:   1185
VmRSS:  18604 kB
Threads: 4

du -sh ~/solemne_so_equipo07 ; df -h /
624K    /home/equipo07/solemne_so_equipo07
/dev/sda1    24G  1.2G  21G   6% /

```

Figura 7. Observación en vivo del proceso concurrente en la consola de la máquina virtual. Es otra corrida de la misma prueba y con la MISMA carga ampliada: los 4000 archivos y 64000 líneas que declara el encabezado de la pantalla son los de la transcripción de arriba. Lo que cambia es la corrida, y por eso difieren el PID, el RSS y el %CPU máximo, que aquí llega a 107 frente a los 103 transcritos. Coincide en lo que importa: 4 hilos vivos, %CPU por encima de 100 y el mismo du -sh de 624K del proyecto.

6.5 Memoria del sistema, espacio disponible y tamaño del proyecto

```

$ free -h ; df -h / ; du -sh ~/solemne_so_equipo07

```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	379Mi	3.3Gi	3.7Mi	373Mi	3.5Gi
Swap:	1.3Gi	0B	1.3Gi			

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	24G	1.2G	21G	6%	/


```

624K    /home/equipo07/solemne_so_equipo07

```

La máquina tiene 3.8Gi de memoria total, con 379Mi en uso y 3.3Gi libres. La columna buff/cache (373Mi) no es memoria perdida sino caché de disco que el núcleo devuelve en cuanto un proceso la necesita, y por eso la disponible (3.5Gi) supera a la libre; el intercambio queda en 0 B usados, señal de que el trabajo nunca presionó la memoria física. El proyecto vive en /dev/sda1 montado en /, con 24G de capacidad, 1.2G usados y 21G disponibles (6% de uso), y ocupa 624K; ese total supera la suma de sus subcarpetas por el redondeo de du a bloques de 4096 bytes.

6.6 Metadatos de la bitácora con stat

```

$ stat logs/gestion_ambiental.log ; ls -lah logs/gestion_ambiental.log

```

```

File: logs/gestion_ambiental.log
Size: 2442          Blocks: 8           IO Block: 4096   regular file
Device: 8,1      Inode: 1046636    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/equipo07)   Gid: ( 1000/equipo07)
Access: 2026-09-09 23:29:11.762321294 -0300
Modify: 2026-09-09 23:29:11.766321401 -0300
Change: 2026-09-09 23:29:11.766321401 -0300
Birth: 2026-09-09 23:29:11.762321294 -0300
-rw-rw-r-- 1 equipo07 equipo07 2.4K Sep  9 23:29 logs/gestion_ambiental.log

```

Campo de stat	Valor	Lectura
Tipo y tamaño	regular file, 2442 B en 8 bloques	Archivo ordinario de datos. El tamaño lógico es menor que el espacio reservado, porque el sistema de archivos asigna bloques completos de 4096 bytes.
Inodo y enlaces	1046636 (dispositivo 8,1), 1 enlace	El nombre gestion_ambiental.log es sólo una entrada de directorio que apunta a ese inodo. Un enlace duro subiría el contador a 2 sin duplicar un byte.
Permisos	0664/-rw-rw-r--	Lectura y escritura para el propietario y su grupo, sólo lectura para el resto: la bitácora es auditable por terceros pero no modificable.
Propietario / grupo	1000/equipo07 / 1000/equipo07	Pertenece al usuario sin privilegios del equipo, no a root: el gestor no necesita permisos elevados para operar.
Access / Modify / Change / Birth	iguales al segundo	Última lectura, última escritura de contenido, último cambio de metadatos del inodo y creación: coinciden porque el archivo se creó y se escribió en la misma corrida.

Esa bitácora tiene 28 líneas y 2442 bytes. Ambas cifras corresponden a la misma corrida: el generador de este informe compara el conteo de líneas del archivo entregado con el que declara la evidencia, y el tamaño del archivo entregado con el que devolvió stat, y se detiene si alguno de los dos pares no calza.

7. Mover frente a copiar, y control de errores

7.1 Por qué los informes se mueven y el resumen se copia

Los informes se MUEVEN desde salida/ a su carpeta de clasificación porque el traslado es un cambio de estado definitivo: un informe ya clasificado no debe seguir figurando como pendiente. Copiarlos duplicaría cada informe y se perdería la propiedad más útil: que salida/ sin informes significa "no queda nada por clasificar".

Mover dentro de una misma partición (aquí origen y destino están ambos en /dev/sda1) es la llamada rename(2): no copia datos, elimina la entrada de directorio antigua y crea otra que apunta al MISMO inodo. Los bloques no se tocan, el número de inodo no cambia y sólo se actualiza el ctime; por eso es casi instantánea y su costo no depende del tamaño. Entre particiones distintas el núcleo no podría renombrar y shutil.move degradaría a copiar y borrar.

El resumen consolidado tiene otro papel: es la vista general que el módulo de monitoreo consulta en su ruta de siempre. Por eso salida/resumen_ambiental.txt se CONSERVA y además se COPIA como gestion_ambiental/resumen_resguardado.txt con shutil.copy. Copiar sí crea un inodo nuevo, con su contenido en otros bloques: desde ese momento los dos archivos son independientes y modificar uno no altera al otro, que es lo que se espera de un respaldo.

	Mover (rename)	Copiar (copy)
Se aplica a	informe_*.txt	resumen_ambiental.txt
Efecto en el inodo	conserva el mismo inodo	crea un inodo nuevo
Datos en disco	no se duplican	se duplican
Original	deja de existir en el origen	permanece en salida/
Propósito	reclasificación definitiva	respaldo y auditoría

Comprobacion posterior a la ejecucion del gestor

```
salida/resumen_ambiental.txt sigue existiendo : SI
  identico a resumen_resguardado.txt          : SI
  informes que quedan en salida/               : 0 (se MOVIERON)
```

El resumen original (448 bytes) sigue en salida/, la copia resguardada pesa 448 bytes y su contenido es idéntico byte a byte: comprobado. En salida/ quedan 0 informes individuales, porque todos fueron movidos y no copiados; esto es lo que documenta 5.4.

7.2 Control de errores sin detener el procesamiento

Esta demostración es DESTRUCTIVA: inyecta datos dañados. Por eso NO se ejecutó sobre el árbol entregado sino sobre una COPIA completa en /home/equipo07/demo_destructiva. De ahí que la bitácora entregada cierre con "no se detectaron anomalías en esta corrida": el entregable viene de una corrida limpia y las cifras de esta subsección pertenecen a la copia.

El requisito 13 pide controlar al menos un error sin detener el procesamiento. Se inyectaron cuatro fallas distintas, para probar tanto el análisis de la bitácora de alertas como la lectura de los informes: una línea sin separadores, otra con campos insuficientes, un indicador desconocido ("Radiacion") y un informe sin la línea "Alertas detectadas: N".

```
$ python3 src/gestor_incidentes.py ; echo "Codigo de salida: $?"
$ grep 'ANOMALIA\|CONTROL DE ERRORES' logs/gestion_ambiental.log
```

Codigo de salida: 0

```
[2026-09-09 23:29:11] ANOMALIA: el informe informe_XXX99_20240101.txt no declara la linea 'Alertas detectadas: N'. No se
clasifica, queda en salida/ para revision manual. Se continua.
[2026-09-09 23:29:11] ANOMALIA: alerta con formato invalido en la linea 80 de alertas/alertas_detectadas.log:
'linea_corrupta_sin_separadores'. Se descarta y se continua.
[2026-09-09 23:29:11] ANOMALIA: alerta con formato invalido en la linea 81 de alertas/alertas_detectadas.log:
'faltan;campos;aqui'. Se descarta y se continua.
[2026-09-09 23:29:11] ANOMALIA: indicador desconocido 'Radiacion' en la linea 82 de alertas/alertas_detectadas.log. Se
descarta y se continua.
CONTROL DE ERRORES: se detectaron 4 anomalias (2 alertas con formato invalido, 1 con indicador desconocido, 0 lineas con
bytes corruptos, 1 informes sin dato de alertas, 0 archivos ignorados en salida, 0 advertencias sobre el origen de
alertas, 0 errores de proceso capturados). Todas quedaron registradas y el procesamiento CONTINUO hasta generar el
inventario.
```

El comportamiento es el correcto: las 4 anomalías quedaron registradas con su número de línea y su contenido, el gestor NO se detuvo, terminó con código 0, conservó las 79 alertas válidas y generó el inventario igual. Descartar el dato defectuoso y continuar es preferible a abortar: una línea corrupta no puede invalidar una jornada de monitoreo, pero tampoco puede desaparecer en silencio.

```

== stat de la bitacora del entregable ==

  File: logs/gestion_ambiental.log
  Size: 2442          Blocks: 8          IO Block: 4096   regular file
Device: 8,1   Inode: 1046636   Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/equipo07)   Gid: ( 1000/equipo07)
Access: 2026-09-09 23:29:11.806322470 -0300
Modify: 2026-09-09 23:29:11.766321401 -0300
Change: 2026-09-09 23:29:11.766321401 -0300
 Birth: 2026-09-09 23:29:11.762321294 -0300

== Control de errores (sobre una copia, para no alterar el entregable) ==
Se inyectaron 2 alertas danadas en alertas_detectadas.log
Codigo de salida del gestor : 0   (no se detuvo)
Anomalias registradas      : 2
Alertas conservadas        : 79

[2026-09-10 12:15:44] CONTROL DE ERRORES: se detectaron 2 anomalias (1 alertas

```

Figura 8. Metadatos de la bitácora con stat y control de errores en la consola de la máquina virtual. Arriba, el stat del archivo entregado: los mismos 2442 bytes e inodo 1046636 que transcribe 6.6. Abajo, una corrida de control DISTINTA de la de 7.2, con dos alertas dañadas en vez de cuatro fallas; sus líneas legibles ya dan el resultado completo (código 0, dos anomalías y 79 alertas conservadas). La última línea, el resumen CONTROL DE ERRORES de esa misma corrida, es la que corta el borde de la consola: no está transcrita en ninguna evidencia, porque la corrida archivada en evidencias/debian/06-control-de-errores.txt es la otra.

7.3 Idempotencia y protección contra sobrescritura

Esta demostración también es destructiva, porque duplica informes a propósito, y por eso se ejecutó sobre la misma COPIA /home/equipo07/demo_destructiva. El árbol entregado conserva sus 20 informes sin sufijos de versión, que es el estado de una única corrida limpia.

```

# Tres corridas consecutivas del gestor sobre la copia
$ ls -lah gestion_ambiental/criticas/informe_STG04_20240101*

corrida 1 -> alertas por indicador: 79 | inventario: 79
corrida 2 -> alertas por indicador: 79 | inventario: 79
corrida 3 -> alertas por indicador: 79 | inventario: 79

-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101.txt
-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101_v1.txt
-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101_v2.txt

```

Las alertas no se duplican entre corridas porque los registros por indicador se reconstruyen de forma atómica sobre archivos temporales que se renombran al final. Al reprocesar, los informes ya clasificados no se pierden ni se pisan: obtener_nombre_seguro() comprueba la existencia del destino y agrega un sufijo correlativo. Partiendo de 21 informes, tras dos ciclos adicionales conviven 61 donde se esperaban 61 (40 con sufijo): ningún archivo fue sobrescrito ni eliminado.

8. Conclusiones

8.1 Sobre concurrencia: el resultado que no favorece la hipótesis

La conclusión más importante es también la más incómoda: en este caso concreto la concurrencia NO aceleró el procesamiento. La versión concurrente tardó 0m0.023s frente a 0m0.019s de la secuencial, y su tiempo interno fue 0.007 s frente a 0.005 s. El dato no se maquilla, se explica:

- El trabajo útil es demasiado pequeño: 20 archivos con 320 líneas, procesados en centésimas de segundo.
- El costo fijo de la concurrencia no depende del tamaño del trabajo: crear 3 hilos, encolar los archivos y tomar y soltar los mutex en cada actualización pesa más que lo que se ahorra al repartir un trabajo tan breve.
- El trabajo está dominado por entrada/salida sobre archivos diminutos que el núcleo ya tiene en caché, de modo que hay poca espera que solapar; y en CPython el bloqueo global del intérprete impide que dos hilos ejecuten código Python puro a la vez.

La concurrencia sí se observa cuando el trabajo crece: con la carga amplificada de 4000 archivos el proceso alcanzó 103 % de CPU, por encima del 100 % que jamás superaría un proceso de un solo hilo. El mecanismo funciona; lo que falta en el conjunto oficial es trabajo suficiente para amortizar su costo de entrada.

8.2 Sobre sincronización y sistema de archivos

Que las 6 de 6 métricas coincidan y que los 20 informes sean idénticos es la evidencia de que la sincronización es correcta: con 3 hilos actualizando contadores compartidos, sin mutex cada corrida habría dado un resultado distinto. El patrón fue acumular en variables locales y entrar a la sección crítica una sola vez por archivo; `queue.Queue` garantiza que cada archivo lo toma un único trabajador. El único efecto observable de la concurrencia es el ORDEN de las líneas de la bitácora.

Mover y copiar obligó a distinguir el nombre de un archivo del archivo mismo: un nombre es una entrada de directorio que apunta a un inodo. Las herramientas completaron el cuadro: `stat` dio inodo, enlaces, permisos y marcas de tiempo; `ls -lah` y `du -sh`, la diferencia entre tamaño lógico y bloques ocupados; `df -h`, el sistema de archivos y su espacio libre; `free -h`, que la caché no es memoria perdida; y `ps` con `/proc`, la estructura interna del proceso.

8.3 Declaraciones de honestidad técnica

- **Hipervisor:** el hito declaró Oracle VirtualBox y la implementación final se hizo sobre Microsoft Hyper-V (máquina de Generación 1, arranque BIOS), con los recursos comprometidos intactos; la pauta admite otro hipervisor. TODAS las figuras de instalación son de la máquina real en Hyper-V: de la etapa previa sobre Oracle VirtualBox no se conserva ninguna captura en el entregable, porque no correspondía a la máquina entregada.
- **Rendimiento:** la versión concurrente resultó más lenta que la secuencial en el conjunto oficial; se informa tal como se midió.
- **Clasificación:** `sin_alertas/` quedó con 0 informes porque la pauta exige al menos una alerta por archivo; la rama se demostró aparte.
- **Demostraciones destructivas:** el control de errores (7.2) y la anti-sobrescritura (7.3) se ejecutaron sobre una COPIA, no sobre el árbol entregado; por eso la bitácora entregada no registra esas anomalías.
- **Trazabilidad:** cada cifra se lee al generar el informe desde evidencias/ y del estado real del proyecto, y el generador compara ambas fuentes -y las cifras del README- antes de escribir nada; si discrepan, no se emite el documento.

8.4 Índice de evidencias que respaldan este informe

Cada sección puede auditarse contra el archivo que la origina; el generador falla si alguno cambia de forma incompatible.

Archivo de evidencia	Contenido	Secciones
evidencias/hyperv/configuracion-vm-hyperv.txt	cmdlets de Hyper-V y verificación SHA256 del ISO	2.1, 2.2
evidencias/debian/01-preparar-ambiente.txt	apt update, apt upgrade y entorno de Python	3
evidencias/debian/02-entorno.txt	distribución, kernel, hipervisor, CPU, memoria, disco y red	2
evidencias/debian/03-ejecucion-parte1.txt	ambas corridas, las seis métricas y la comparación de informes y alertas	4
evidencias/debian/04-observacion-procesos.txt	<code>/usr/bin/time -v, ps, ps -L, /proc, free -h, df -h</code> y <code>du -sh</code>	6
evidencias/debian/05-gestor-incidencias.txt	estructura, inventario, <code>stat</code> de la bitácora, mover frente a copiar	5, 7.1
evidencias/debian/06-control-de-errores.txt	anomalías inyectadas y rama <code>sin_alertas</code> (sobre una copia)	5.3, 7.2
evidencias/debian/07-anti-sobrescritura.txt	sufijos <code>_v1</code> y <code>_v2</code> e idempotencia (sobre una copia)	7.3
evidencias/salida_parte1/	21 archivos: <code>salida/</code> tal como la dejó la Parte 1	5.4
evidencias/fotos/	instalación real en Hyper-V y consola de la máquina definitiva	Figuras 1 a 8
<code>gestion_ambiental/</code> , <code>alertas/</code> , <code>logs/</code> , <code>salida/</code>	estado real: informes, alertas por indicador, inventario y bitácora	5, 7
docs/Solemne01PracticaParte2FormaB.pdf	pauta oficial contra la cual se verificó cada requisito	todas